

Collections

Course: Rust Basics

Author: Aleksandr Kugushev

Vec

```
struct Vec<T, A: Allocator = Global> {  
    buf: RawVec<T, A>,  
    len: usize,  
}
```

```
struct RawVec<T, A: Allocator =  
Global> {  
    ptr: Unique<T>,  
    cap: usize,  
    alloc: A,  
}
```

```
struct Unique<T: ?Sized> {  
    pointer: NonNull<T>,  
    _marker: PhantomData<T>,  
}
```

```
struct NonNull<T: ?Sized> {  
    pointer: *const T,  
}
```

```
struct PhantomData<T: ?Sized>;
```

Vec

```
let mut vec: Vec<i32> = Vec::new();  
vec.push(42);  
let last: Option<i32> = vec.pop();
```

Vec

```
let mut vec = vec![1, 2, 3];
```

```
let count = vec.len();
```

```
let first:i32 = vec[0];
```

```
let mut vec = vec![String::from("Hi")];
```

```
let first: String = vec[0];
```

error[E0507]: cannot move out of index of `Vec<String>`

Get by reference

```
let mut vec = vec![String::from("Hi")];
```

```
let first: &String = &vec[0];
```

```
let first: Option<&String> = vec.get(0);
```

Borrow element, borrow whole vector

```
let mut vec = vec![String::from("Hi")];
```

```
let first: &String = &vec[0];
```

```
vec.push(String::from("World"));
```

```
println!("{}", first);
```

error[E0502]: cannot borrow `vec` as mutable because it is also borrowed as immutable

Set vec element

```
let first_mut: &mut String = &mut vec[0];  
*first_mut = String::from("Hello");
```

```
vec[0] = String::from("World");
```

Drain

```
let mut vec = vec![1, 2, 3, 4, 5, 6, 7];  
let drained: Drain<i32> = vec.drain(2..5);  
drop(drained);  
println!("{}", vec.len()) // 4
```


Range

```
let arr = [0, 1, 2, 3, 4];
assert_eq!(arr[ .. ], [0, 1, 2, 3, 4]);
assert_eq!(arr[ .. 3], [0, 1, 2   ]);
assert_eq!(arr[ ..=3], [0, 1, 2, 3  ]);
assert_eq!(arr[1.. ], [ 1, 2, 3, 4]);
assert_eq!(arr[1.. 3], [ 1, 2   ]); // This is a `Range`
assert_eq!(arr[1..=3], [ 1, 2, 3  ]);

for i in 0..42 {
    println!("{}", i)
}
```

Range

```
pub struct Range<Idx> {  
    /// The lower bound of the range (inclusive).  
    pub start: Idx,  
    /// The upper bound of the range (exclusive).  
    pub end: Idx,  
}
```

Apply Range on array

```
let arr = [0, 1, 2, 3, 4];
```

```
let s: [i32] = arr[..];
```

```
let i = 1;
```

```
let b = arr[i..];
```

error[E0277]: the size for values of type `[i32]` cannot be known at compilation time

Slice

```
let mut vec = vec![1, 2, 3, 4, 5, 6, 7];  
let slice: &[i32] = &vec[2..5];
```

Slice is ref

```
let mut vec = vec![1, 2, 3, 4, 5, 6, 7];  
let slice: &[i32] = &vec[2..5]; // 3  
drop(vec);  
println!("{:?}", slice);
```

error[E0505]: cannot move out of `vec` because it is borrowed

Slice

```
let mut vec = vec![1, 2, 3, 4, 5, 6, 7];  
let slice: &i32 = &vec[2..5];  
let count = slice.len(); // 3  
println!("{}", count);  
let item: &i32 = &slice[1]; // 4  
let item: Option<&i32> = slice.get(1);
```

Slice mut

```
let mut vec = vec![1, 2, 3, 4, 5, 6, 7];  
let mut slice_mut: &mut [i32] = &mut vec[2..5];  
slice_mut[0] = 42;  
println!("{}", vec[2]) // 42
```

Slice without &

```
let arr = [0, 1, 2, 3, 4];
```

```
let s: [i32] = arr[..];
```

error[E0277]: the size for values of type `[integer]` cannot be known at compilation time

Boxed slice

```
let vec = vec![1, 2, 3, 4, 5, 6, 7];  
let boxed: Box<i32> = vec.into_boxed_slice();  
let slice = &boxed[2..5];  
println!("{:?}", slice);
```

Split at

```
let vec = vec![1, 2, 3, 4, 5, 6, 7];  
let (left, right): (&[i32], &[i32]) = vec.split_at(3);  
println!("{:?} {:?}", left, right);
```

Practice: Slice

```
let vec = vec!['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd'];
```

- Get Slice with “world” and print it
- Get 2 slices: “hello” and “world”
- Replace “world” with “space”
- Replace with: `let mut vec = String::from("hello world");`
- Replace "hello world" with "привет мир"

String

```
pub struct String {  
    vec: Vec<u8>,  
}  
  
let string_eng = String::from("Hello!!");  
let string_china = String::from("读写汉字学中文");  
println!("{}", string_eng.len()); // 7  
println!("{}", string_china.len()); // 21
```

String slice aka &str

```
let string = String::from("Hello World");  
let slice: &str = &string[..];  
let other_slice = "Hello World";
```

Slice UTF-8

```
let string_china = String::from("读写汉字学中文");
```

```
let slice_china = &string_china[1..2];
```

```
println!("{}", slice_china);
```

panic: byte index 1 is not a char boundary; it is inside '读' (bytes 0..3) of `读写汉字学中文`

```
let slice_chars: Vec<char> = string_china.chars().collect();
```

```
let chars_china: &[char] = &slice_chars[1..2];
```

```
println!("{}", chars_china[0]); // 写
```

Replace char at index

```
let mut string = String::from("Hello World");  
let slice_mut: &mut str = &mut string[1..2];  
*slice_mut[0..1] = "00";  
error[E0614]: type `str` cannot be dereferenced
```

```
string.replace_range(1..2, "00");  
println!("{}", string); // H00llo World
```

HashMap

```
let mut scores = HashMap::new();  
scores.insert(String::from("Blue"), 10);  
scores.insert(String::from("Yellow"), 50);  
scores.remove("Blue");  
let contains: bool = scores.contains_key("Yellow");  
let score: Option<i32> = scores.get("Yellow");
```


HashMap Entry API

```
scores.entry(String::from("Yellow")).or_insert(42);  
match scores.entry(String::from("Yellow")) {  
    Entry::Occupied(x) => {  
        println!("{}", x.key(), x.get()) // Yellow 50  
    }  
    Entry::Vacant(x) => {  
        println!("{}", x.key())  
    }  
}
```

For

```
let vector = vec![1, 2, 3, 4, 5, 6, 7, 8, 9];  
for x in vector {  
    println!("{}", x);  
}  
for x in vector {  
    println!("{}", x);  
}
```

error[E0382]: use of moved value: `vector`

consider iterating over a slice of the `Vec<i32>`'s content to avoid moving into the `for` loop

For

```
let vector = vec![1, 2, 3, 4, 5, 6, 7, 8, 9];  
for x in &vector {  
    println!("{}", x);  
}  
for x in &vector {  
    println!("{}", x);  
}
```

Iterators

```
let vector = vec![1, 2, 3, 4, 5, 6, 7, 8, 9]; let result: Vec<String> =  
vector
```

```
let mut result =  
Vec::<String>::new();
```

```
for i in 0..vector.len() {  
    if vector[i] > 5 {  
        result.push(i.to_string())  
    }  
}
```

```
.iter()  
.filter(|x| { **x > 5 })  
.map(|x| { x.to_string() })  
.collect();
```

Iterator

```
trait Iterator {  
    type Item;  
  
    fn next(&mut self) -> Option<Self::Item>;  
}
```

Iterator

```
let vector = vec![1, 2, 3, 4, 5, 6, 7, 8, 9];  
let mut iter = vector.iter();  
loop {  
    let option: Option<&i32> = iter.next();  
    match option {  
        None => { break; }  
        Some(x) => { println!("{}", x); }  
    }  
}
```

Iterator

```
let vector = vec![1, 2, 3, 4, 5, 6, 7, 8, 9];
let mut iter = vector.iter()
    .filter(|x| { **x > 5 })
    .map(|x| { x.to_string() });
loop {
    let option: Option<String> = iter.next();
    match option {
        None => { break; }
        Some(x) => { println!("{}", x); }
    }
}
```

For

```
let mut iter = vector.iter();
loop {
    let option = iter.next();
    match option {
        None => { break; }
        Some(x) => { println!("{}", x); }
    }
}
```

```
for x in vector.iter() {
    println!("{}", x);
}
```


For and Intolterator trait

```
for x in vector.iter() {  
    println!("{}", x);  
}  
  
for x in vector {  
    println!("{}", x);  
}
```

```
trait Intolterator {  
    type Item;  
    type Intolter: Iterator<Item =  
Self::Item>;  
    fn into_iter(self) -> Self::Intolter;  
}
```

Iter vs IterMut vs IntoIterator

```
impl<T> [T] {  
    pub fn iter(&self) -> Iter<'_, T> {  
        Iter::new(self)  
    }  
  
    pub fn iter_mut(&mut self) ->  
    IterMut<'_, T> {  
        IterMut::new(self)  
    }  
}
```

```
pub trait IntoIterator {  
    type Item;  
    type Intolter: Iterator<Item =  
Self::Item>;  
    fn into_iter(self) ->  
Self::Intolter;  
}
```

Why into_iter requires trait?

*/// Creates a consuming iterator, that is, one that moves each value out of
/// the vector (from start to end). The vector cannot be used after calling
/// this.*

```
fn into_iter(self) -> Self::IntoIter {  
    unsafe {  
        let mut me = ManuallyDrop::new(self);  
        let alloc = ManuallyDrop::new(ptr::read(me.allocator()));  
        let begin = me.as_mut_ptr();  
        let end = if T::IS_ZST {  
            begin.wrapping_byte_add(me.len())  
        } else {  
            begin.add(me.len()) as *const T  
        };  
        let cap = me.buf.capacity();  
        IntoIter {  
            buf: NonNull::new_unchecked(begin),  
            phantom: PhantomData,  
            cap,  
            alloc,  
            ptr: begin,  
            end,  
        }  
    }  
}
```

Iterator functions

https://danielkeep.github.io/itercheat_baked.html

- Generators: once, repeat, etc.
- Sequences: map, filter, flat_map, etc.
- Collectors: collect, find, max, sum, any, etc.

Generators

- `my_vec.iter()`
- `iter::once(1)`
- `iter::repeat(4)` // repeat forever
- `iter::empty::<i32>()`

Sequences

map , filter , flat_map , flatten , zip , unzip , chain , take , skip ,
enumerate, filter_map, cloned ...

Map

```
let vector = vec![1, 2, 3, 4, 5, 6, 7, 8, 9];  
let map_result = vector.iter().map(|x| x.to_string());
```

```
fn map<B, F>(self, f: F) -> Map<Self, F>  
where  
    Self: Sized,  
    F: FnMut(Self::Item) -> B,  
{  
    Map::new(self, f)  
}
```

Map

```
fn map<B, F>(self, f: F) -> Map<Self, F>
where
  Self: Sized,
  F: FnMut(Self::Item) -> B,
{
  Map::new(self, f)
}
```

```
pub struct Map<I, F> {
  iter: I,
  f: F,
}

impl<B, I: Iterator, F> Iterator for Map<I, F>
where F: FnMut(I::Item) -> B,
{
  type Item = B;

  #[inline]
  fn next(&mut self) -> Option<B> {
    match self.iter.next() {
      Some(x) => Some(self.f(x)),
      None => None,
    }
  }
}
```


Filter

```
let vector = vec![1, 2, 3, 4, 5, 6, 7, 8, 9];  
let result = vector.iter()  
    .filter(|x| **x > 5)  
    .map(|x| x.to_string());
```

Filter

```
fn filter<P>(self, predicate: P) -> Filter<Self, P>
where
    Self: Sized,
    P: FnMut(&Self::Item) -> bool,
{
    Filter::new(self, predicate)
}
```

```
pub struct Filter<I, P> {
    iter: I,
    predicate: P,
}

impl<I: Iterator, P> Iterator for Filter<I, P>
where P: FnMut(&I::Item) -> bool,
{
    type Item = I::Item;

    #[inline]
    fn next(&mut self) -> Option<I::Item> {
        self.iter.find(&mut self.predicate)
    }
}
```

Iterators are lazy

```
let vector = vec![1, 2, 3, 4, 5, 6, 7, 8, 9];
```

```
vector.iter()  
  .filter(|x| **x > 5)  
  .map(|x| x.to_string());
```

```
fn next(&mut self) -> Option<B> {  
  match self.iter.next() {  
    Some(x) => Some(self.f(x)),  
    None => None,  
  }  
}
```

```
fn next(&mut self) -> Option<I::Item> {  
  self.iter.find(&mut self.predicate)  
}
```

Iterators are lazy

```
let result: Map<Filter<Iter<i32>, fn(&&i32) -> bool>, fn(&i32) ->
String> = vector.iter()
    .filter(|x| **x > 5)
    .map(|x| x.to_string());
for item in result {
    println!("{}", item)
}
```



```
fn next(&mut self) -> Option<I::Item> {
    self.iter.find(&mut self.predicate)
}
```

```
fn next(&mut self) -> Option<B> {
    match self.iter.next() {
        Some(x) => Some(self.f(x)),
        None => None,
    }
}
```

Zero Cost Abstractions: Iterator

Java

Number of elements	100	1,000	10,000	100,000	1,000,000
Numbers per element	100	1,000	10,000	10,000	1,000
Number of runs	50,000	5,000	50	10	10
Average time when using streams on arrays	1.116 ms	94.37 ms	8.44 s	1 m 25 s	1 m 26.6 s
Average time when using streams on lists	1.287 ms	137.85 ms	18.2 s	OOM occurred	Not even gonna bother
Average time when using loops on arrays	292.924 us	3487 ms	4.525 s	43.44 s	33.63

Rust

test bench_search_for ... bench: 19,620,300 ns/iter (+/- 915,700)

test bench_search_iter ... bench: 19,234,900 ns/iter (+/- 657,200)

<https://doc.rust-lang.org/book/ch13-04-performance.html>

Sequences

- flat_map
- zip
- take , skip
- filter_map
- cloned

Sorting 0_o?

- <https://crates.io/crates/itertools>

```
fn sorted_by<F>(self, cmp: F) -> VecIntolter<Self::Item>
    where Self: Sized,
           F: FnMut(&Self::Item, &Self::Item) -> Ordering,
{
    let mut v = Vec::from_iter(self);
    v.sort_by(cmp);
    v.into_iter()
}
```

Iterators and ownership

- `iter()`
- `iter_mut()`
- `into_iter()`

iter_mut()

```
let mut vector = vec![1, 2, 3, 4];  
let result = vector.iter_mut()  
    .filter(|x| **x > 2);  
for item in result {  
    *item = *item + 10;  
}  
for x in vector {  
    println!("{}", x) // 1 2 13 14  
}
```

into_iter()

```
let vector = vec![1, 2, 3, 4, 5, 6, 7, 8, 9];
```

```
let result = vector.into_iter()  
    .filter(|x| *x > 5)  
    .map(|x| x.to_string());
```

```
for item in result {  
    println!("{}", item)  
}
```

```
for x in vector {  
    println!("{}", x)  
}
```

error[E0382]: use of moved value: `vector`

Collectors

```
let vector = vec![1, 2, 3, 4, 5, 6, 7, 8, 9];
```

```
let result: Vec<String> = vector.iter()  
    .filter(|x| **x > 5)  
    .map(|x| x.to_string())  
    .collect();
```

collect()

```
let vector = vec![1, 2, 3, 4, 5, 6, 7, 8, 9];
```

```
let result = vector.iter()  
    .filter(|x| **x > 5)  
    .map(|x| x.to_string())  
    .collect();
```

error[E0282]: type annotations needed

collect()

```
fn collect<B: FromIterator<Self::Item>>(self) -> B
where
    Self: Sized,
{
    FromIterator::from_iter(self)
}
```

```
pub trait FromIterator<A>: Sized {
    fn from_iter<T: IntoIterator<Item = A>>(iter: T) -> Self;
}
```

Iterator

```
pub trait Iterator {  
    /// The type of the elements being iterated over.  
    type Item;  
  
    /// Advances the iterator and returns the next value.  
    fn next(&mut self) -> Option<Self::Item>;  
}
```

Iterator for Fibonacci

```
impl Iterator for Fibonacci {  
    type Item = u32;  
    fn next(&mut self) -> Option<Self::Item> {  
        let current = self.curr;  
        self.curr = self.next;  
        self.next = current + self.next;  
        Some(current)  
    }  
}
```

Homework

- Rustlings:
 - iterators
 - vecs
 - strings
 - Hashmaps
- Courseware
 - Refactoring: use iterators